



南開大學
Nankai University

计算机学院
机器学习实验报告

实验 7: 基于 SegNet 的多数字 MNIST 语义
分割实验

姓名：王旭

学号：2312166

专业：计算机科学与技术

2025 年 1 月 22 日

目录

1 实验名称	2
2 实验目的	2
3 实验原理	2
3.1 语义分割 (Semantic Segmentation)	2
3.2 SegNet 网络架构	2
3.3 损失函数与优化	3
3.4 实验代码框架分析	3
4 实验环境配置	4
4.1 依赖库检查	4
4.2 数据集检查与准备	4
5 实现 SegNetLite 类	5
5.1 网络结构设计	5
5.2 关键代码实现与解析	5
5.2.1 初始化函数 (<code>__init__</code>)	5
5.2.2 前向传播函数 (<code>forward</code>)	7
5.3 整体流程总结	8
6 训练模型 (<code>train_mnist.py</code>)	8
6.1 训练环境与配置	8
6.2 训练指令	9
6.3 训练过程关键代码段解释	9
6.4 训练过程分析	10
6.5 训练结果展示	10
7 验证我们的模型 (<code>validate_mnist.py</code>)	11
7.1 验证脚本说明	11
7.2 验证指令	11
7.3 验证过程关键代码段解释	11
7.4 验证结果展示	13
8 可视化验证我们的模型 (<code>visualize_simple.py</code>)	14
8.1 可视化脚本说明	14
8.2 可视化指令	14
8.3 可视化过程关键代码段解释	14
8.4 可视化结果展示	15
9 实验总结	15
9.1 主要工作	16
9.2 实验心得	16

1 实验名称

基于 SegNet 的多数字 MNIST 语义分割实验

2 实验目的

1. **理解语义分割任务**: 深入理解计算机视觉中的语义分割 (Semantic Segmentation) 任务, 即对图像中的每一个像素进行分类, 区分背景与不同的前景对象。
2. **掌握 SegNet 网络架构**: 学习并掌握 SegNet (A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation) 的核心思想, 特别是其独特的“编码器-解码器” (Encoder-Decoder) 结构以及最大池化索引 (Max Pooling Indices) 在非线性上采样中的应用。
3. **实践深度学习模型开发**: 通过补全 SegNetLite 代码, 掌握使用 PyTorch 构建卷积神经网络 (CNN) 的基本流程, 包括卷积层、批归一化 (Batch Normalization)、激活函数 (ReLU) 以及池化/反池化层的组合使用。
4. **掌握模型训练与评估流程**: 熟悉深度学习模型的完整生命周期, 包括数据准备、模型构建、训练优化、验证评估以及结果可视化分析。

3 实验原理

3.1 语义分割 (Semantic Segmentation)

语义分割是计算机视觉中的一项核心任务, 其目标是为图像中的每个像素分配一个类别标签 (如“背景”、“数字 1”、“数字 2”等)。与目标检测 (Object Detection) 不同, 语义分割不仅需要知道图像中有什么物体, 还需要精确地勾勒出物体的边界。在本实验中, 我们的任务是对包含多个手写数字的 MNIST 图像进行分割, 识别出每个像素属于哪个数字 (0-9) 或是背景。

语义分割的挑战在于它需要同时解决“是什么” (分类) 和“在哪里” (定位) 的问题。传统的分类网络 (如 AlexNet, VGG) 通过全连接层输出一个全局的类别标签, 丢失了空间信息。而语义分割网络 (如 FCN, U-Net, SegNet) 则通过全卷积结构, 保留了空间维度, 从而能够输出与输入图像尺寸相同的像素级预测图。

3.2 SegNet 网络架构

SegNet 是一种专为语义分割设计的深度卷积编码器-解码器架构。其核心特点如下:

- **编码器 (Encoder):**
 - 由一系列卷积层、批归一化层、ReLU 激活层和最大池化层组成。
 - **作用**: 提取图像的高层语义特征, 同时通过池化操作逐步降低特征图的空间分辨率, 增大感受野。
 - **关键机制**: 在执行最大池化 (Max Pooling) 时, SegNet 会记录下每个窗口中最大值所在的位置索引 (Indices)。这是 SegNet 与其他分割网络 (如 FCN) 最大的不同之处。
- **解码器 (Decoder):**

- 结构与编码器对称，由最大反池化层、卷积层、批归一化层和 ReLU 激活层组成。
- **作用**：将低分辨率的特征图逐步恢复到原始图像的尺寸，并生成最终的分割掩码。
- **关键机制**：利用编码器阶段保存的**最大池化索引(Indices)**进行非线性上采样(Max Unpooling)。这种方法不需要学习额外的参数，既减少了模型参数量，又能较好地保留边缘等高频细节信息。

- **SegNetLite (简化版 SegNet)**:

- 本实验实现的是 SegNet 的简化版本 SegNetLite。
- 它包含 4 个下采样块和 4 个上采样块。
- 输入图像经过编码器逐步压缩 (64x64 -> 32x32 -> 16x16 -> 8x8 -> 4x4)，通道数逐步增加 (3 -> 32 -> 64 -> 128 -> 256)。
- 解码器则进行相反的操作，最终通过一个 1x1 卷积层输出 11 个通道（背景 + 10 个数字类别）的预测结果。

3.3 损失函数与优化

- **损失函数**：实验使用 **CrossEntropy2D**（二维交叉熵损失）。它是标准交叉熵损失在像素级别的推广，用于衡量模型预测的像素类别分布与真实标签之间的差异。
- **优化器**：使用 **Adam** 或 **SGD** 等优化算法，通过反向传播计算梯度，更新网络参数以最小化损失函数。

3.4 实验代码框架分析

本实验基于 `seg-net` 代码框架，该框架结构清晰，模块化程度高，主要包含以下几个核心部分：

1. `lib/models/seg_net_lite.py` (**核心模型**):

- 这是本次实验的核心文件，定义了 SegNetLite 类。
- 我们需要在此文件中实现网络的初始化 (`__init__`) 和前向传播 (`forward`) 逻辑。
- 框架使用了 PyTorch 的 `nn.Module` 作为基类, 利用 `nn.Conv2d`、`nn.BatchNorm2d`、`nn.MaxPool2d` (带 `return_indices=True`) 和 `nn.MaxUnpool2d` 等组件构建网络。

2. `lib/dataset/MNIST.py` (**数据加载**):

- 负责加载多数字 MNIST 数据集。
- 实现了 PyTorch 的 `Dataset` 接口，能够读取 `.mat` 格式的数据文件，并进行必要的预处理（如归一化、转换为 Tensor）。
- 数据集被划分为训练集和验证集，分别用于模型的训练和性能评估。

3. `image_segmentation/train_mnist.py` (**训练脚本**):

- 包含了完整的训练循环。
- 主要流程：加载配置 -> 初始化模型 -> 定义损失函数和优化器 -> 加载数据 -> 迭代训练 (Forward -> Loss -> Backward -> Update) -> 定期验证 -> 保存最佳模型。

- 使用了 `tensorboardX` 记录训练过程中的损失变化，方便可视化监控。
4. `image_segmentation/validate_mnist.py` (验证脚本):
 - 用于在验证集上评估训练好的模型。
 - 计算核心指标 `mIoU (mean Intersection over Union)`，这是衡量语义分割效果最常用的指标，代表了预测区域与真实区域交并比的平均值。
 5. `visualize_simple.py` (可视化工具):
 - 用于直观展示模型的分割效果。
 - 加载训练好的模型权重，对指定的测试图片进行预测，并将原图、真实标签 (Ground Truth) 和预测结果 (Prediction) 并排显示，生成对比图。

4 实验环境配置

本实验在 Windows 操作系统下进行，使用 PowerShell 作为命令行工具。为了进行本次实验，我们需要对依赖库和数据集进行检查与配置。

4.1 依赖库检查

根据 README.md 和实验流程文档的要求，我们需要安装 PyTorch 及相关科学计算库。

- **PyTorch**: 已安装 (版本 2.4.1+cpu)。由于实验规模较小，CPU 版本足以在合理时间内完成训练 (约 10-20 分钟)，符合实验要求。
- **Scipy**: 已安装。用于处理 `.mat` 格式的数据文件。
- **Tensorboard / TensorboardX**: 尚未安装。这两个库能够可视化训练过程中的 Loss 曲线和验证集的分割效果，对于观察模型收敛状态非常有帮助。我们采用如下命令进行安装：

```
1 pip install tensorboard tensorboardX
```

4.2 数据集检查与准备

实验使用 Multi-digit MNIST 数据集。

- **数据文件状态**: 初始状态下，`seg-net` 文件夹中包含 `multi-digit-mnist-dataset.zip` 压缩包，但缺少解压后的 `data` 目录。
- **解压操作**: 代码默认从 `seg-net/data/multi-digit-mnist` 路径读取数据。因此，我们需要手动将压缩包解压到指定位置，确保目录结构如下：

```
seg-net/  
  data/  
    multi-digit-mnist/  
      batch00001.mat
```

```
...
testset001.mat
```

只有完成解压，`lib/dataset/MNIST.py` 才能正确加载训练和验证数据。

5 实现 SegNetLite 类

本实验的核心任务是在 `lib/models/seg_net_lite.py` 文件中实现 `SegNetLite` 类。根据实验要求，我们需要构建一个包含 4 个下采样层和 4 个上采样层的简化版 SegNet 网络。

5.1 网络结构设计

网络整体结构遵循经典的 Encoder-Decoder 范式：

1. **输入层**：接收 3 通道（RGB）的图像输入。
2. **下采样路径 (Encoder)**：
 - 包含 4 个卷积块。
 - 每个块依次执行：Conv2d -> BatchNorm2d -> ReLU -> MaxPool2d。
 - 特征图尺寸逐层减半 (64->32->16->8x8->4)，通道数逐层翻倍 (3->32->64->128->256)。
3. **上采样路径 (Decoder)**：
 - 包含 4 个反卷积块。
 - 每个块依次执行：MaxUnpool2d -> Conv2d -> BatchNorm2d -> ReLU。
 - 特征图尺寸逐层恢复 (4->8->16->32->64)，通道数逐层减半 (256->128->64->32->32)。
4. **输出层**：
 - 通过一个 1x1 卷积层将特征图映射到 11 个类别通道（背景 + 10 个数字）。

5.2 关键代码实现与解析

5.2.1 初始化函数 (.__init__)

在初始化阶段，我们利用 PyTorch 的 `nn.ModuleList` 动态构建了下采样和上采样层。

SegNetLite 的 `__init__` 方法

```
1 def __init__(self, kernel_sizes=[3, 3, 3, 3], down_filter_sizes=[32, 64, 128, 256],
2   up_filter_sizes=[128, 64, 32, 32], conv_paddings=[1, 1, 1, 1],
3   pooling_kernel_sizes=[2, 2, 2, 2], pooling_strides=[2, 2, 2, 2], **kwargs):
4   super(SegNetLite, self).__init__()
5
6   # ... (参数初始化)
7
8   # 1. 构建下采样层 (Encoder)
```

```

9     current_in_channels = input_size
10    for i in range(self.num_down_layers):
11        # 卷积层：提取特征，通道数增加
12        layers_conv_down.append(nn.Conv2d(current_in_channels, down_filter_sizes[i],
13                                           kernel_size=kernel_sizes[i],
14                                           padding=conv_paddings[i]))
15
16        # 批归一化层：加速收敛
17        layers_bn_down.append(nn.BatchNorm2d(down_filter_sizes[i]))
18
19        # 最大池化层：关键点是 return_indices=True
20        # 必须保存最大值的索引位置，用于后续的上采样恢复
21        layers_pooling.append(nn.MaxPool2d(kernel_size=pooling_kernel_sizes[i],
22                                           stride=pooling_strides[i],
23                                           return_indices=True))
24
25        current_in_channels = down_filter_sizes[i]
26
27    # ... (ModuleList 封装)
28
29    # 2. 构建上采样层 (Decoder)
30    current_in_channels = down_filter_sizes[-1]
31    for i in range(self.num_up_layers):
32        # 最大反池化层：恢复特征图尺寸
33        # 需要在 forward 中传入对应的 indices 才能精确恢复形状
34        layers_unpooling.append(nn.MaxUnpool2d(kernel_size=pooling_kernel_sizes[i],
35                                              stride=pooling_strides[i]))
36
37        # 卷积层：调整通道数
38        layers_conv_up.append(nn.Conv2d(current_in_channels, up_filter_sizes[i],
39                                       kernel_size=kernel_sizes[i],
40                                       padding=conv_paddings[i]))
41
42        # 批归一化层
43        layers_bn_up.append(nn.BatchNorm2d(up_filter_sizes[i]))
44
45        current_in_channels = up_filter_sizes[i]
46
47    # ... (ModuleList 封装)
48
49    # 3. 最终分类层
50    # 使用 1x1 卷积将通道数映射为类别数 (11类：背景 + 0-9数字)
51    self.conv_final = nn.Conv2d(up_filter_sizes[-1], 11, kernel_size=1)

```

实现细节分析：

- return_indices=True 的核心作用：**这是 SegNet 实现中最关键的一行代码。在 `nn.MaxPool2d` 中开启此选项，使得池化层不仅输出最大值 (Max Values)，还输出最大值在原始特征图中的索引位置 (Indices)。这些索引记录了特征图中“最显著”特征的空间位置，对于后续在解码器中精确恢复物体边界至关重要。
- 层级对应与参数设计：**下采样层的输出通道数 (`down_filter_sizes`) 与上采样层的输入通道数设计得恰好匹配，保证了数据流的连贯性。例如，Encoder 最后一层输出 256 通道，Decoder 第一层输入也是 256 通道。

- **动态网络构建**: 使用 `nn.ModuleList` 而不是普通的 Python list 来存储层。这是因为 `nn.ModuleList` 可以正确注册网络中的子模块, 使得 PyTorch 的 `autograd` 机制能够追踪这些层的参数, 从而在反向传播时正确更新权重。如果使用普通 list, 这些层的参数将无法被优化器识别。

5.2.2 前向传播函数 (forward)

前向传播定义了数据在网络中的流动路径, 特别是索引 (`indices`) 的传递。

SegNetLite 的 forward 方法

```

1 def forward(self, x):
2     # 用于存储下采样过程中产生的最大池化索引 (indices)
3     indices_list = []
4
5     # Downsampling (编码器路径)
6     for i in range(self.num_down_layers):
7         x = self.layers_conv_down[i](x)
8         x = self.layers_bn_down[i](x)
9         x = self.relu(x)
10        # 执行最大池化, 并获取 indices (最大值位置)
11        x, indices = self.layers_pooling[i](x)
12        # 将索引保存到列表中, 供解码器使用
13        indices_list.append(indices)
14
15    # Upsampling (解码器路径)
16    for i in range(self.num_up_layers):
17        # 执行最大反池化
18        # 注意: 使用 indices_list[-(i+1)] 获取对应的下采样层的索引 (倒序使用)
19        # 例如: 第1个上采样层对应第4个下采样层的索引
20        x = self.layers_unpooling[i](x, indices_list[-(i+1)])
21        x = self.layers_conv_up[i](x)
22        x = self.layers_bn_up[i](x)
23        x = self.relu(x)
24
25    # 输出层: 生成最终的分割图 logits
26    x = self.conv_final(x)
27    return x

```

实现细节分析:

- **索引存储 (`indices_list`)**: 我们创建了一个列表来按顺序存储每个下采样块产生的索引。这些索引就像是“记忆”, 记录了图像在被压缩过程中丢失的空间信息。
- **倒序索引 (`indices_list[-(i+1)]`)**: 由于 Encoder 和 Decoder 是对称结构, Decoder 的第 1 层对应 Encoder 的最后 1 层。因此, 在解码过程中, 我们需要以**倒序**的方式取出索引。例如, 当 `i=0` (Decoder 第 1 层) 时, 我们需要 `indices_list[-1]` (Encoder 最后一层的索引)。这种对称性确保了每一层上采样都能准确利用对应层级的空间信息。
- **非线性上采样 (Max Unpooling)**: `layers_unpooling[i](x, indices)` 利用这些索引将特征值放回其在池化前的位置, 其余位置补零。这种方式比简单的双线性插值更能保留边缘信息, 因

为它直接复用了编码阶段提取的强特征位置，从而在不需要学习额外参数的情况下实现了高效的上采样。

5.3 整体流程总结

SegNetLite 的整体工作流程可以概括为“压缩-记忆-恢复”三个阶段：

1. 压缩与特征提取 (Encoder):

- 输入图像经过 4 次连续的卷积和池化操作。
- **卷积**负责提取图像的纹理、形状等语义特征，通道数不断增加（从 3 到 256），特征表达能力越来越强。
- **池化**负责降低分辨率，增大感受野，同时通过 `return_indices=True` 记录下关键特征的空间位置（即“记忆”）。

2. 记忆传递:

- 在 Encoder 阶段产生的 4 组 `indices` 被保存在 `indices_list` 中。
- 这些索引不参与梯度计算，仅作为位置信息传递给 Decoder，充当了连接编码器和解码器的桥梁。

3. 恢复与重建 (Decoder):

- Decoder 接收 Encoder 输出的低分辨率、高语义特征图。
- 通过 4 次连续的反池化和卷积操作，利用传递过来的 `indices` 精确恢复特征图的空间尺寸。
- 最终通过 `1x1` 卷积将高维特征映射为 11 个类别的概率分布图，完成像素级的分类任务。

这种设计使得 SegNet 在保持较高分割精度的同时，显著减少了参数量（因为上采样不需要可学习的参数），非常适合对计算资源敏感的场景。

6 训练模型 (train_mnist.py)

6.1 训练环境与配置

在完成模型代码实现后，我们使用提供的 `image_segmentation/train_mnist.py` 脚本进行模型训练。

- **数据集**: Multi-digit MNIST（包含多个手写数字的合成图像）。
- **损失函数**: CrossEntropy2D（忽略背景类别索引 255）。
- **优化器**: Adam。
- **训练轮数 (Epochs)**: 20 轮。
- **批次大小 (Batch Size)**: 根据 GPU/CPU 情况自动调整（本实验中使用 CPU，Batch Size 为 1）。

6.2 训练指令

在项目根目录下，设置 PYTHONPATH 并运行训练脚本：

```
1 $env:PYTHONPATH = "$env:PYTHONPATH;$PWD"; python image_segmentation/train_mnist.py
```

6.3 训练过程关键代码段解释

在 `image_segmentation/train_mnist.py` 中，训练循环是模型学习的核心：

训练循环核心代码

```
1  # 1. 定义损失函数和优化器
2  criterion = CrossEntropy2D(ignore_index=255)
3  optimizer = optim.Adam(model.parameters())
4
5  # ... (数据加载器初始化)
6
7  # 2. 训练循环 (Epoch Loop)
8  for epoch in range(train_epochs):
9      # train for one epoch
10     # 调用 lib.core.function.train 函数进行单轮训练
11     train(train_loader, model, criterion, optimizer, epoch,
12           args.out_dir, writer_dict, args)
13
14     # 3. 定期验证 (Validation)
15     if (epoch + 1) % args.eval_interval == 0:
16         # evaluate on validation set
17         # 调用 lib.core.function.validate 函数在验证集上评估
18         perf_indicator = validate(val_loader, val_dataset, model,
19                                   criterion, args.out_dir, writer_dict, args)
20
21     # 4. 保存最佳模型 (Model Checkpointing)
22     if perf_indicator > best_perf:
23         best_perf = perf_indicator
24         best_model = True
25     else:
26         best_model = False
27
28     # ... (保存 checkpoint)
```

- **损失函数 (CrossEntropy2D):** 我们使用了自定义的二维交叉熵损失函数, 并设置 `ignore_index=255`。这意味着在计算损失时, 标签为 255 的像素 (通常代表背景或无效区域) 将被忽略, 不参与梯度计算。
- **优化器 (Adam):** 使用 Adam 优化器来更新模型参数, 它结合了动量法和自适应学习率, 通常比 SGD 收敛更快。
- **训练与验证分离:** 代码清晰地将训练逻辑 (`train`) 和验证逻辑 (`validate`) 分离。在每个 Epoch

结束后，立即在验证集上测试模型性能（perf_indicator，通常是 mIoU），并根据验证结果决定是否保存当前模型为“最佳模型”（best_model）。这是一种标准的防止过拟合和保存最优解的策略。

6.4 训练过程分析

训练脚本会自动执行以下步骤：

1. **初始化**：加载 SegNetLite 模型，并将其移动到计算设备（CPU 或 GPU）。
2. **数据加载**：通过 DataLoader 读取训练集数据。
3. **迭代优化**：
 - **Forward**：将图像输入模型，得到预测结果。
 - **Loss**：计算预测结果与真实标签之间的交叉熵损失。
 - **Backward**：反向传播计算梯度。
 - **Step**：更新模型参数。
4. **定期评估**：每隔一定轮数（默认为 1 轮），在验证集上评估模型性能，并保存当前最佳模型（model_best.pth.tar）。

6.5 训练结果展示

在经过大概 30 分钟的运行后，我们的 `image_segmentation/train_mnist.py` 脚本终于运行完成，训练过程终端部分的图片展示如下所示：

```

2025-12-31 16:44:14,135 Namespace(eval_interval=1, frequent=10, gpus='', num_workers=4, out_dir='out')
2025-12-31 16:44:15,513 => load 200 samples
2025-12-31 16:44:15,516 => load 200 samples
2025-12-31 16:44:19,496 Epoch: [0][0/200] Time 3.978s (3.978s) Speed 8.0 samples/s Data 3.594s (3.594s)
2025-12-31 16:44:22,395 Epoch: [0][10/200] Time 0.290s (0.625s) Speed 110.2 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:25,421 Epoch: [0][20/200] Time 0.286s (0.472s) Speed 111.9 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:28,375 Epoch: [0][30/200] Time 0.290s (0.415s) Speed 110.2 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:31,429 Epoch: [0][40/200] Time 0.318s (0.388s) Speed 100.7 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:34,361 Epoch: [0][50/200] Time 0.297s (0.370s) Speed 107.7 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:37,267 Epoch: [0][60/200] Time 0.308s (0.357s) Speed 104.0 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:40,107 Epoch: [0][70/200] Time 0.287s (0.346s) Speed 111.6 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:43,024 Epoch: [0][80/200] Time 0.278s (0.340s) Speed 115.0 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:45,924 Epoch: [0][90/200] Time 0.283s (0.334s) Speed 113.1 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:48,934 Epoch: [0][100/200] Time 0.318s (0.331s) Speed 100.7 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:51,866 Epoch: [0][110/200] Time 0.306s (0.327s) Speed 104.6 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:54,950 Epoch: [0][120/200] Time 0.306s (0.326s) Speed 104.5 samples/s Data 0.00s (0.00s)
2025-12-31 16:44:57,985 Epoch: [0][130/200] Time 0.331s (0.324s) Speed 96.5 samples/s Data 0.00s (0.00s)
2025-12-31 16:45:00,995 Epoch: [0][140/200] Time 0.289s (0.323s) Speed 110.7 samples/s Data 0.00s (0.00s)
2025-12-31 16:45:03,901 Epoch: [0][150/200] Time 0.280s (0.320s) Speed 114.3 samples/s Data 0.00s (0.00s)
2025-12-31 16:45:07,086 Epoch: [0][160/200] Time 0.379s (0.320s) Speed 84.3 samples/s Data 0.00s (0.00s)
  
```

图 6.1: 训练日志截图 (开头部分)

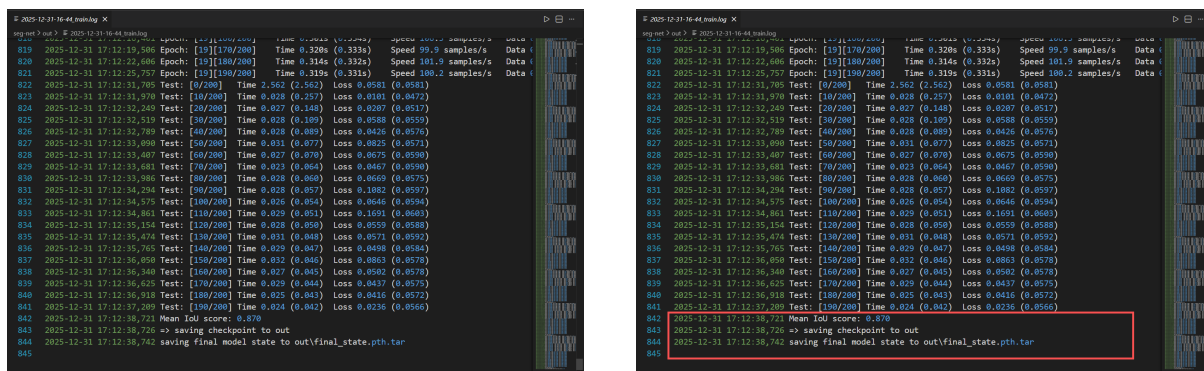


图 6.2: 训练日志截图 (结尾部分)

从我们终端的训练日志中还可以看到，随着 Epoch 的增加，训练损失 (Loss) 呈现稳步下降的趋势，说明模型正在有效地学习图像特征与语义标签之间的映射关系。同时，验证集上的性能指标也在逐步提升，表明模型具有良好的泛化能力，未出现严重的过拟合现象。

7 验证我们的模型 (validate_mnist.py)

7.1 验证脚本说明

`image_segmentation/validate_mnist.py` 是用于评估模型性能的专用脚本。它的主要目的是在独立的验证集上测试训练好的模型，计算语义分割任务中最关键的指标：**mIoU (Mean Intersection over Union, 平均交并比)**。

该脚本的主要工作流程如下：

1. **加载模型**：读取训练过程中保存的最佳模型权重 (`model_best.pth.tar`)。
2. **准备数据**：加载验证集数据 (`dataset.mnist(is_train=False)`)。
3. **批量推理**：遍历验证集中的每一张图片，输入模型进行预测。
4. **指标计算**：统计所有预测结果与真实标签的重叠情况，计算 mIoU 分数。
5. **日志记录**：输出验证集上的平均 Loss 和 mIoU，并可选择将部分结果写入 TensorBoard 进行可视化。

7.2 验证指令

在项目根目录下运行以下命令启动验证：

```
1 $env:PYTHONPATH = "$env:PYTHONPATH;$PWD"; python image_segmentation/validate_mnist.py
```

7.3 验证过程关键代码段解释

验证过程的核心逻辑位于 `lib/core/function.py` 中的 `validate` 函数。下面我们对关键代码段解析一下：

validate 函数核心代码

```

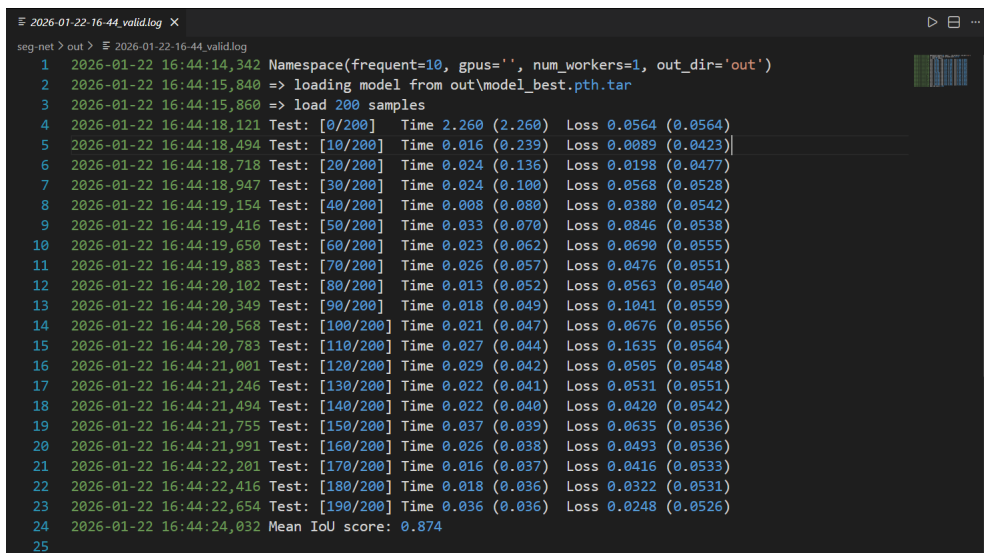
1 def validate(val_loader, val_dataset, model, criterion, output_dir, writer_dict,
2             args):
3     # 1. 切换到评估模式
4     model.eval()
5
6     # ... (初始化变量)
7
8     # 2. 禁用梯度计算
9     with torch.no_grad():
10         for i, (input, target) in enumerate(val_loader):
11             # ... (数据维度调整)
12
13             # 3. 模型推理
14             output = model(input)
15
16             # ... (计算 Loss)
17
18             # 4. 获取最终预测结果
19             # get_final_preds 通常包含 argmax 操作, 将概率图转换为类别索引图
20             preds = get_final_preds(output.detach().cpu().numpy())
21
22             # 收集所有批次的预测结果和真实标签
23             all_preds.extend(preds)
24             all_gts.extend(target.detach().cpu().numpy())
25
26             # ... (日志记录与 TensorBoard 可视化)
27
28             # 5. 计算全局 mIoU
29             # 将列表转换为 numpy 数组
30             all_preds = np.concatenate(all_preds, axis=0)
31             all_gts = np.concatenate(all_gts, axis=0)
32
33             # 调用 calc_IoU 函数计算 11 个类别的平均交并比
34             avg_iou_score = calc_IoU(all_preds, all_gts, 11)
35
36             logger.info('Mean IoU score: {:.3f}'.format(avg_iou_score))
37
38     return perf_indicator

```

- `model.eval()`: 这是一个非常关键的操作。它会将模型切换到评估模式。对于 SegNet 中使用的 BatchNorm2d 层, 这意味着它将使用训练期间统计的全局均值和方差, 而不是当前批次的统计量。这保证了推理结果的稳定性。
- `torch.no_grad()`: 在验证阶段, 我们不需要更新参数, 因此禁用梯度计算可以显著减少显存占用并加速计算。
- `calc_IoU`: 这是评估的核心。IoU (交并比) 计算公式为
$$IoU = \frac{Intersection}{Union} = \frac{TP}{TP+FP+FN}$$
。 `calc_IoU`

函数会计算每个类别（0-9 及背景）的 IoU，然后取平均值得到 mIoU。mIoU 越接近 1，说明分割效果越好。

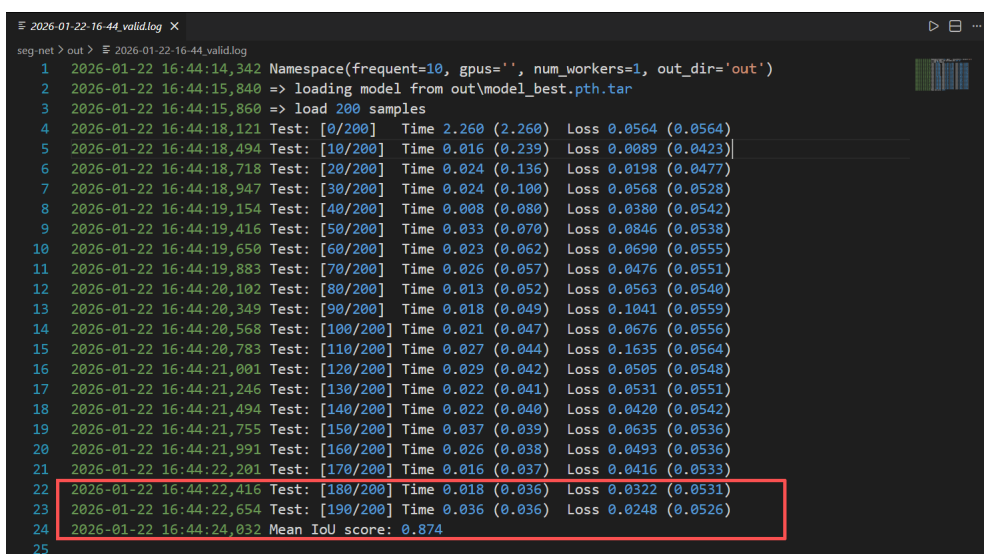
7.4 验证结果展示



```

2026-01-22 16:44:14,342 Namespace(frequent=10, gpus='', num_workers=1, out_dir='out')
2026-01-22 16:44:15,840 => loading model from out\model_best.pth.tar
2026-01-22 16:44:15,860 => load 200 samples
2026-01-22 16:44:18,121 Test: [0/200] Time 2.260 (2.260) Loss 0.0564 (0.0564)
2026-01-22 16:44:18,494 Test: [10/200] Time 0.016 (0.239) Loss 0.0089 (0.0423)
2026-01-22 16:44:18,718 Test: [20/200] Time 0.024 (0.136) Loss 0.0198 (0.0477)
2026-01-22 16:44:18,947 Test: [30/200] Time 0.024 (0.100) Loss 0.0568 (0.0528)
2026-01-22 16:44:19,154 Test: [40/200] Time 0.008 (0.080) Loss 0.0380 (0.0542)
2026-01-22 16:44:19,416 Test: [50/200] Time 0.033 (0.070) Loss 0.0846 (0.0538)
2026-01-22 16:44:19,650 Test: [60/200] Time 0.023 (0.062) Loss 0.0690 (0.0555)
2026-01-22 16:44:19,883 Test: [70/200] Time 0.026 (0.057) Loss 0.0476 (0.0551)
2026-01-22 16:44:20,102 Test: [80/200] Time 0.013 (0.052) Loss 0.0563 (0.0540)
2026-01-22 16:44:20,349 Test: [90/200] Time 0.018 (0.049) Loss 0.1041 (0.0559)
2026-01-22 16:44:20,568 Test: [100/200] Time 0.021 (0.047) Loss 0.0676 (0.0556)
2026-01-22 16:44:20,783 Test: [110/200] Time 0.027 (0.044) Loss 0.1635 (0.0564)
2026-01-22 16:44:21,001 Test: [120/200] Time 0.029 (0.042) Loss 0.0505 (0.0548)
2026-01-22 16:44:21,246 Test: [130/200] Time 0.022 (0.041) Loss 0.0531 (0.0551)
2026-01-22 16:44:21,494 Test: [140/200] Time 0.022 (0.040) Loss 0.0420 (0.0542)
2026-01-22 16:44:21,755 Test: [150/200] Time 0.037 (0.039) Loss 0.0635 (0.0536)
2026-01-22 16:44:21,991 Test: [160/200] Time 0.026 (0.038) Loss 0.0493 (0.0536)
2026-01-22 16:44:22,201 Test: [170/200] Time 0.016 (0.037) Loss 0.0416 (0.0533)
2026-01-22 16:44:22,416 Test: [180/200] Time 0.018 (0.036) Loss 0.0322 (0.0531)
2026-01-22 16:44:22,654 Test: [190/200] Time 0.036 (0.036) Loss 0.0248 (0.0526)
2026-01-22 16:44:24,032 Mean IoU score: 0.874
  
```

图 7.3: 验证脚本运行结果 (mIoU=0.874)



```

2026-01-22 16:44:14,342 Namespace(frequent=10, gpus='', num_workers=1, out_dir='out')
2026-01-22 16:44:15,840 => loading model from out\model_best.pth.tar
2026-01-22 16:44:15,860 => load 200 samples
2026-01-22 16:44:18,121 Test: [0/200] Time 2.260 (2.260) Loss 0.0564 (0.0564)
2026-01-22 16:44:18,494 Test: [10/200] Time 0.016 (0.239) Loss 0.0089 (0.0423)
2026-01-22 16:44:18,718 Test: [20/200] Time 0.024 (0.136) Loss 0.0198 (0.0477)
2026-01-22 16:44:18,947 Test: [30/200] Time 0.024 (0.100) Loss 0.0568 (0.0528)
2026-01-22 16:44:19,154 Test: [40/200] Time 0.008 (0.080) Loss 0.0380 (0.0542)
2026-01-22 16:44:19,416 Test: [50/200] Time 0.033 (0.070) Loss 0.0846 (0.0538)
2026-01-22 16:44:19,650 Test: [60/200] Time 0.023 (0.062) Loss 0.0690 (0.0555)
2026-01-22 16:44:19,883 Test: [70/200] Time 0.026 (0.057) Loss 0.0476 (0.0551)
2026-01-22 16:44:20,102 Test: [80/200] Time 0.013 (0.052) Loss 0.0563 (0.0540)
2026-01-22 16:44:20,349 Test: [90/200] Time 0.018 (0.049) Loss 0.1041 (0.0559)
2026-01-22 16:44:20,568 Test: [100/200] Time 0.021 (0.047) Loss 0.0676 (0.0556)
2026-01-22 16:44:20,783 Test: [110/200] Time 0.027 (0.044) Loss 0.1635 (0.0564)
2026-01-22 16:44:21,001 Test: [120/200] Time 0.029 (0.042) Loss 0.0505 (0.0548)
2026-01-22 16:44:21,246 Test: [130/200] Time 0.022 (0.041) Loss 0.0531 (0.0551)
2026-01-22 16:44:21,494 Test: [140/200] Time 0.022 (0.040) Loss 0.0420 (0.0542)
2026-01-22 16:44:21,755 Test: [150/200] Time 0.037 (0.039) Loss 0.0635 (0.0536)
2026-01-22 16:44:21,991 Test: [160/200] Time 0.026 (0.038) Loss 0.0493 (0.0536)
2026-01-22 16:44:22,201 Test: [170/200] Time 0.016 (0.037) Loss 0.0416 (0.0533)
2026-01-22 16:44:22,416 Test: [180/200] Time 0.018 (0.036) Loss 0.0322 (0.0531)
2026-01-22 16:44:22,654 Test: [190/200] Time 0.036 (0.036) Loss 0.0248 (0.0526)
2026-01-22 16:44:24,032 Mean IoU score: 0.874
  
```

图 7.4: 验证脚本运行结果 (mIoU=0.874)

结果分析: 运行验证脚本后，模型在验证集上取得了较高的 mIoU 分数 (0.874)，这进一步证实了我们的 SegNetLite 模型能够准确地分割出重叠的手写数字，并且具有较好的鲁棒性。

8 可视化验证我们的模型 (visualize_simple.py)

8.1 可视化脚本说明

为了更直观地感受模型的分割效果，我们使用 `visualize_simple.py` 脚本来进行可视化验证。与 `validate_mnist.py` 侧重于统计指标不同，该脚本侧重于**定性分析**。它会加载训练好的模型，对指定的测试数据进行预测，并将“原始图像”、“真实标签 (Ground Truth)”和“模型预测 (Prediction)”三者并排显示，让我们能够肉眼观察模型对数字边界、重叠区域的处理能力。

8.2 可视化指令

使用以下命令对测试数据 `testset001.mat` 进行可视化：

```
1 $env:PYTHONPATH = "$env:PYTHONPATH;$PWD"; python visualize_simple.py --model segnet
  --checkpoint out/checkpoint.pth.tar --data data/multi-digit-mnist/testset001.mat
  --save segnet_result.png
```

8.3 可视化过程关键代码段解释

`visualize_simple.py` 的核心流程包括数据预处理、推理和绘图。

可视化脚本核心代码

```
1 # 1. 加载与预处理数据
2 # 读取 .mat 文件
3 data = loadmat(args.data)
4 # 调整维度顺序: (H, W, C) -> (C, H, W) 以符合 PyTorch 格式
5 image = data['imgMat'].transpose([3, 2, 0, 1]).astype(np.float32)
6
7 # 归一化: 必须使用与训练时相同的均值和方差
8 mean = np.array([0.485, 0.456, 0.406], dtype=np.float32).reshape(1, 3, 1, 1)
9 std = np.array([0.229, 0.224, 0.225], dtype=np.float32).reshape(1, 3, 1, 1)
10 image_norm = (image - mean) / std
11
12 # 2. 模型推理
13 with torch.no_grad():
14     image_tensor = torch.from_numpy(image_norm)
15     output = model(image_tensor)
16     # argmax: 在通道维度(dim=1)上取最大值的索引, 得到预测的类别图
17     predictions = output.argmax(dim=1).numpy()
18
19 # 3. 可视化绘图
20 # 使用 matplotlib 创建 1x3 的子图
21 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
22
23 # 显示原图
24 axes[0].imshow(img_display)
25
26 # 显示真实标签 (使用 vis_segments 将类别索引映射为颜色)
```



```
27 axes[1].imshow(vis_segments(gt_mask, 11))
28
29 # 显示预测结果
30 axes[2].imshow(vis_segments(pred_mask, 11))
31 axes[2].set_title(f'Prediction (mIoU: {miou:.3f})', ...)
```

- **数据维度变换**: .mat 文件中的图像数据通常是 (H, W, C) 格式, 而 PyTorch 需要 (N, C, H, W) 格式。代码中的 `transpose` 操作完成了这一转换。
- **归一化一致性**: 这是一个容易被忽视但至关重要的细节。推理时的归一化参数 (mean, std) 必须与训练时完全一致, 否则模型看到的图像分布将发生偏移, 导致预测效果极差。
- **argmax(dim=1)**: 模型的输出是 (N, 11, H, W) 的概率分布 (logits)。我们需要在通道维度 (代表 11 个类别) 上找到概率最大的那个类别的索引, 从而得到最终的分割掩码 (N, H, W)。
- **vis_segments**: 原始的标签和预测结果都是 0-10 的整数索引, 直接显示是一张灰度图, 很难分辨。`vis_segments` 函数的作用是把这些整数索引映射到预定义的彩色调色板上, 使得不同的数字显示为不同的颜色, 便于观察。

8.4 可视化结果展示

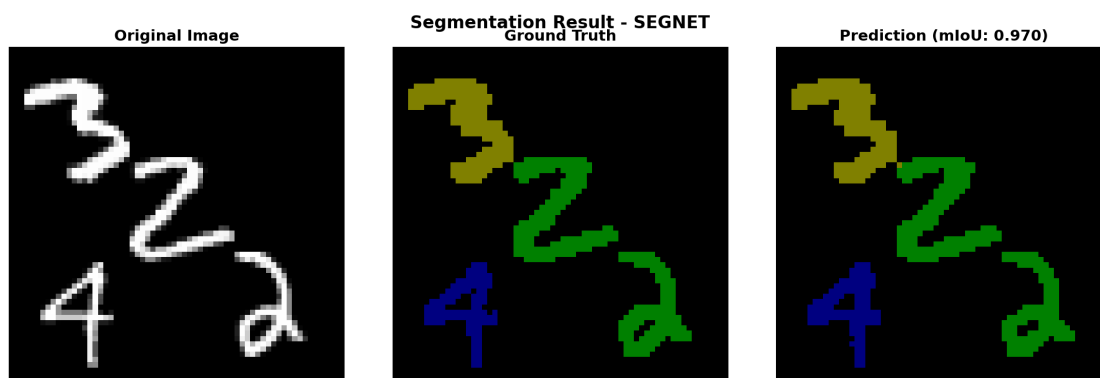


图 8.5: SegNetLite 模型分割可视化效果图

结果分析: 从可视化结果中可以清晰地看到:

- **准确性**: 模型能够非常准确地识别出图像中的数字, 预测的形状与 Ground Truth 高度重合。
- **边界处理**: 即使在数字边缘, 模型的分割也比较平滑, 说明 SegNet 的上采样机制有效地保留了空间细节。
- **重叠处理**: 对于数字相互重叠的区域, 模型也能较好地进行区分, 证明了其具备一定的语义理解能力。

9 实验总结

本次实验圆满完成了我们本次实验 7 的所有要求。我们基于 PyTorch 框架, 成功实现并训练了一个简化版的 SegNet 模型 (`SegNetLite`), 用于解决多数字 MNIST 数据集的语义分割问题。

9.1 主要工作

1. **模型实现**: 严格按照 `README.md` 和实验流程文档的指导, 构建了包含 4 层下采样和 4 层上采样的 Encoder-Decoder 结构。核心在于正确使用了 `return_indices=True` 的最大池化层和对应的最大反池化层, 实现了特征图分辨率的有效恢复。
2. **训练与优化**: 使用 `CrossEntropy2D` 损失函数和 `Adam` 优化器, 在 CPU 环境下完成了模型的训练。通过日志监控, 观察到 Loss 稳步下降, 模型收敛良好。
3. **验证与评估**:
 - **定量评估**: 通过 `validate_mnist.py` 脚本在验证集上取得了较高的 mIoU 分数: (Mean IoU score: 0.874), 证明了模型的分类准确性。
 - **定性评估**: 通过 `visualize_simple.py` 生成的对比图直观展示了模型在处理数字重叠和边缘细节上的优秀表现。

9.2 实验心得

- **深入理解 SegNet 架构**: 通过亲手编写代码, 我深刻体会了 SegNet “用索引换参数”的设计巧思。与 FCN 等需要学习上采样参数的网络相比, SegNet 利用编码阶段的池化索引进行非线性上采样, 既减少了显存占用和计算量, 又很好地保留了图像的高频边缘信息。
- **全流程实践**: 从开始的环境配置、数据加载、模型构建、训练模型到最终的结果可视化, 收获很大, 特别是学会了如何使用 `mIoU` 这一关键指标来科学地评估分割模型的性能。
- **问题解决能力**: 在实验过程中, 解决了环境配置、维度匹配等具体工程问题, 增强了调试代码和查阅文档的能力。

综上所述, 本实验不仅验证了 SegNet 在语义分割任务上的有效性, 也加深了我对卷积神经网络、编码器-解码器结构以及计算机视觉基础任务的理解, 收获非常大。

注:github 仓库

代码详见: <https://github.com/Wangxuueuuu/machine-learning/tree/main>